

MEDIATEK

MT8676 Yocto System User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Kai Peng	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	4
List of Tables	4
1 System	5
1.1 Abbreviations	5
1.2 Yocto System Architecture	6
1.2.1 Software Architecture	6
1.2.2 Yocto Meta Layers	6
1.2.3 Code Structure	7
1.2.4 Common File Paths	9
1.2.5 Partition Table	10
1.3 Yocto Compilation and Flashing	12
1.3.1 Build Server	12
1.3.2 Compiling Software	12
1.3.3 Software Composition	13
1.3.4 Flashing Software	14
1.3.5 Compile SDK	16
1.3.6 Use SDK	16
1.4 Yocto Customization	17
1.4.1 Common Variables	17
1.4.2 Basic Structure of a bb File	18
1.4.3 Boot on Startup	19
1.4.4 Dependencies	20
1.4.5 Out-of-tree Drivers	21
1.4.6 image bb	23
1.4.7 Introduction to fog	23
1.4.8 Create Project	24
1.4.9 Create Meta Layer	24
1.5 Yocto Build Mode	25
1.5.1 Yocto Build Mode Usage	26
1.5.2 Yocto Build Mode Adaptation	26
1.6 Yocto Debugging Techniques	27
1.6.1 Build Directory Structure	27
1.6.2 Commonly Used Subdirectories in the Work Directory	28
1.6.3 Printing Variable Values	29
1.6.4 Accelerating Yocto Compilation	30
1.7 Boot Process	31
1.7.1 BootROM	31
1.7.2 LK2	31
1.7.3 Kernel	32
1.7.4 initramfs	33
1.7.5 systemd	33

1.8 Common Debugging Tools in Yocto	34
Exhibit 1 Terms and Conditions	35

List of Figures

Figure 1-1. Yocto software architecture	6
Figure 1-2. MT8676 meta layers	6
Figure 1-3. Code structure	8
Figure 1-4. Download SP_Flash_Tool	14
Figure 1-5. Load flash.xml	15
Figure 1-6. Flashing Software	15
Figure 1-7. Installing SDK	16
Figure 1-8. SDK installation directory structure	17
Figure 1-9. meta layer directory structures	25
Figure 1-10. After Compilation Directory Structure	28
Figure 1-11. Boot Process	31
Figure 1-12. kernel_init	32

List of Tables

Table 1-1. Abbreviations	5
Table 1-2. meta layer Description	7
Table 1-3. Main module path	8
Table 1-4. Common File Paths	9
Table 1-5. Partition Table Format	9
Table 1-6. Ko Table Format	10
Table 1-7. Partition Table	10
Table 1-8. build server requirements	12
Table 1-9. Image File Descriptions	13
Table 1-10. Yocto Common Variables	17
Table 1-11. create project	24
Table 1-12. Yocto build mode	25
Table 1-13. build variant	26
Table 1-15. Commonly Used Files and Directories in the Build Directory	28
Table 1-16. Commonly Used Subdirectories in the Work Directory	29
Table 1-16. Common Services	33
Table 1-17. Common Debugging Tools in Yocto	34

1 System

This section mainly introduces the MT8676 Yocto system-related content and methods for dealing with common system issues.

1.1 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
bb	Bitbake recipe file, a bb file describes how to build a specific software package.
Bitbake	The core build tool of the Yocto project, which parses bb and bbclass files and executes tasks.
CCU	Camera Control Unit
DPM	DRAM power manager, processing instructions from SPM, controlling DRAM speed, and managing entry/exit from low power states.
layer	Also known as a metadata layer, used by Yocto to manage bb and configuration files. Layers can be developed and maintained independently.
MCUPM	CPU-specific power/performance manager.
NeuroPilot	NeuroPilot ¹ is a set of software tools and APIs for developing efficient AI applications on the MediaTek platform.
OE	OpenEmbedded, which provides the essential components for building embedded Linux systems, including common toolchains, libraries, and applications.
Poky	The reference distribution of the Yocto project. It is a complete build system used to create embedded Linux images.
SCP	System Companion Processor, a subsystem that performs always-on tasks while the system is in a low power state.
SPM	System power management
SSPM	Secure System Power Manager
T-box	Telematics-BOX
tinysys	Refers to processors other than the main processor, such as APUSYS.
VCP	Video Controller Processor
Yocto	An open-source project that enables the creation of custom Linux distributions for embedded systems.

¹ <https://neuropilot.mediatek.inc/>

1.2 Yocto System Architecture

1.2.1 Software Architecture

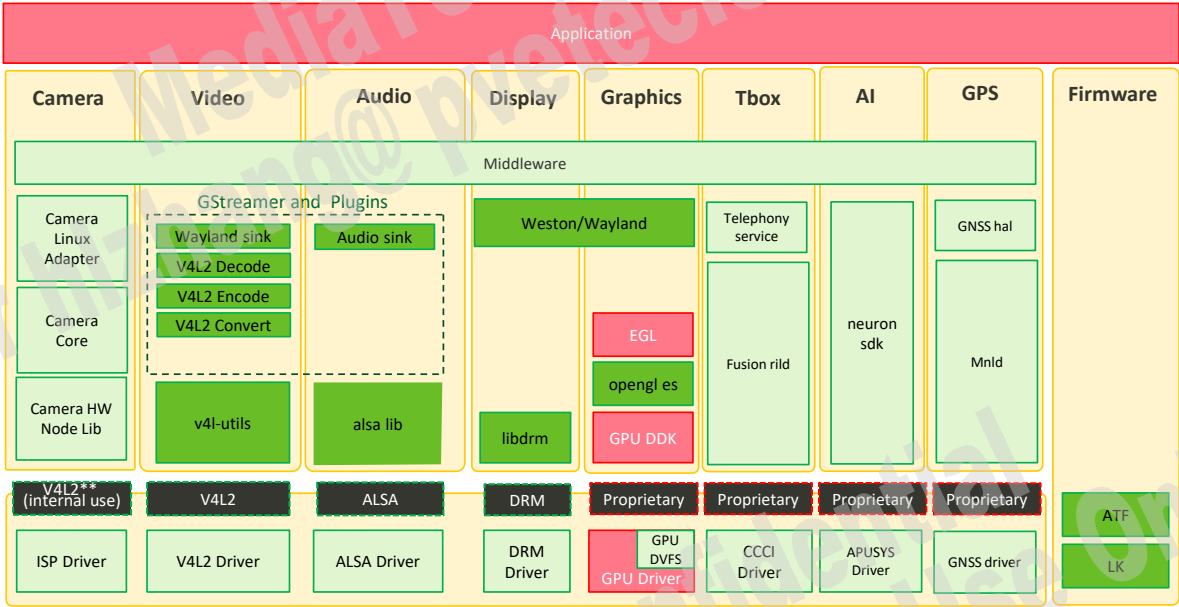


Figure 1-1. Yocto software architecture

1.2.2 Yocto Meta Layers

The MT8676 Yocto meta layers, as shown in Figure 1-2, are primarily composed of four parts: the Yocto core layer, the OSS layer, the MediaTek feature layer, and the MediaTek platform layer. Table 1-2 provides detailed descriptions of the contents included in each layer.

```
BBLAYERS ?= " \
    ##OEROOT##/./meta-mediatek-classes-overlay \
    ##OEROOT##/meta \
    ##OEROOT##/meta-poky \
    ##OEROOT##/meta-yocto-bsp \
    ##OEROOT##/./meta-openembedded/meta-fileystems \
    ##OEROOT##/./meta-openembedded/meta-python \
    ##OEROOT##/./meta-openembedded/meta-multimedia \
    ##OEROOT##/./meta-openembedded/meta-networking \
    ##OEROOT##/./meta-openembedded/meta-oe \
    ##OEROOT##/./meta-clang \
    ##OEROOT##/./meta-mediatek \
    ##OEROOT##/./meta-mediatek-gpl \
    ##OEROOT##/./meta-mediatek-gplv2 \
    ##OEROOT##/./meta-mediatek-gstreamer \
    ##OEROOT##/./meta-mediatek-ml-np \
    ##OEROOT##/./meta-mediatek-mt8676 \
    ##OEROOT##/./meta-qt5 \
"
```

Figure 1-2. MT8676 meta layers

Table 1-2. Meta layer description

Layer	Explanation
meta-mediatek-classes-overlay	Overlays Yocto bbclasses, for custom bbclasses.
meta ²	Contains the core metadata of OpenEmbedded, does not include distribution releases, only provides support for simulators.
meta-poky	Contains the configuration and metadata for the Poky distribution release.
meta-yocto-bsp ³	Contains the BSP for reference hardware of the Yocto Project.
meta-fileystems	Contains metadata related to file systems, such as fuse, owfs, ntfs-3g, etc.
meta-python	Contains metadata related to Python.
meta-multimedia	Contains metadata related to multimedia.
meta-networking	Contains metadata related to networking.
meta-oe	Other shared OE metadata
meta-clang	Contains metadata related to clang and llvm.
meta-qt5	Contains metadata related to qt5.
meta-mediatek	The basic MediaTek layer, including bbclasses and platform-independent packages written by MediaTek.
meta-mediatek-gpl	The MediaTek GPL layer, including packages such as u-boot.
meta-mediatek-gplv2	The MediaTek GPLv2 layer, including GPLv2 versions of packages such as grep.
meta-mediatek-gstreamer	The MediaTek GStreamer layer
meta-mediatek-ml-np	Contains packages related to MediaTek NeuroPilot.
meta-mediatek-mt8676	The MT8676 BSP layer, containing configuration files and metadata for the MT8676 BSP.

1.2.3 Code Structure

The MT8676 Yocto code structure is as shown in Figure 1-3.

- The meta directory contains all the Yocto metadata, for detailed explanation please refer to Section 1.2.2.
- The patch directory contains patches applied by MediaTek to OSS.
- The prebuilt directory contains precompiled files, such as executables, dynamic libraries, images, and toolchains, etc.
- The src directory contains all the source code files.

Table 1-3 lists the paths of the main modules and their brief descriptions.

² <https://git.openembedded.org/openembedded-core/tree/README.OE-Core.md>

³ <https://git.yoctoproject.org/poky/tree/meta-yocto-bsp/README.hardware.md>

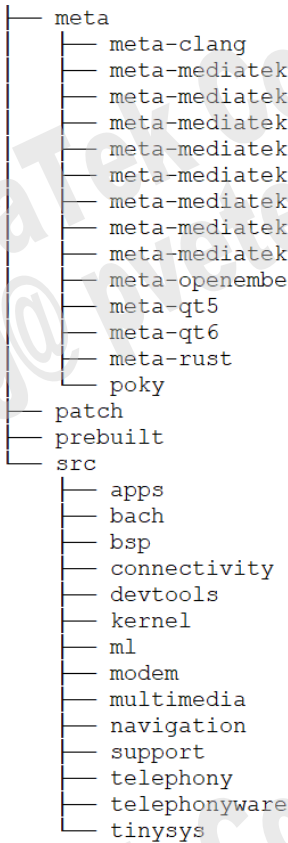


Figure 1-3. Code structure

Table 1-3. Main module path

Layer	Explanation
meta/meta-mediatek-mt8676	MT8676 platform layer
src/bsp/lk2 src/bsp/dramk_8676	bootloader source code
src/kernel/linux/v6.1_mt8676/co_common	Linux Kernel 6.1 source code
src/kernel/linux/v6.1_mt8676/co_device_module	MTK Linux kernel 6.1 device drivers
src/kernel/modules	out-of-tree device drivers
src/apps/spm-base src/apps/atom-base	Application source code
src/connectivity	Wi-Fi related source code
src/ml/neuropilot	NeuroPilot ⁴ related source code
src/telephony src/telephonyware	Source code related to car T-Box (telephony) data/call/SMS firmware.
src/tinysys	Source code related to tinysys

⁴ <https://neuropilot.mediatek.inc/>

1.2.4 Common File Paths

Table 1-4 lists some common file paths in the Yocto system and their functions.

Table 1-4. Common file paths

Path	Explanation
meta/meta-mediatek-mt8676/conf/machine/auto8676p1_64.conf	Project configuration file
meta/meta-mediatek-mt8676/recipes-bsp/ptgen/files/auto8676p1_64/partition_table_emmc_ab.csv	Partition table. The format of the partition table is referred to in Section 1.2.4.1 .
meta/meta-mediatek-mt8676/recipes-kernel/linux/ko_order_table/auto8676p1_64/ko_order_table.csv	ko table, determining the loading order of ko. The format is referred to in Section 1.2.4.2 .
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/auto8676p1_64.dts	kernel dts file
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/configs/auto8676p1_64_defconfig	kernel defconfig file
src/devtools/dct/dws/mt6897/auto8676p1_64.dws	dws file, mainly used for configuring GPIO
meta-mediatek-mt8676/recipes-core/base-files/base-files/auto8676p1_64/fstab	fstab file, controlling partition mounting
meta-mediatek-mt8676/recipes-auto/images/mtk-core-image-auto8676.bb	MT8676 image bb, used to generate the root filesystem

1.2.4.1 Partition Table Format

The partition table is a CSV file that can be edited with a text editor or Excel. For an explanation of each column, please refer to [Table 1-5](#).

Table 1-5. Partition table format

Column Header	Description
Partition_Name	Partition name
Type	Partition type, EXT4 or Raw data
Size_KB	Partition size in kilobytes (KB)
Region(emmc)	The eMMC physical partition where the partition is located, EMMC_BOOT1, EMMC_BOOT2, or EMMC_USER.
Region(ufs)	The UFS physical partition where the partition is located, UFS_LU0, UFS_LU1 (BOOT), or UFS_LU2 (USER).
Reserved	Reserved partition, Y (Yes) or N (No).
Download	Whether to flash the partition, Y (Yes) or N (No). If Y, Download_File cannot be empty.
Download_File	The filename to be flashed
OTA_Update	Whether the partition is updated during OTA (Over-The-Air) update.
EmptyBoot_Needed	Not used

Column Header	Description
Operation_Type	BOOTLOADERS: The preloader partition AUTO: A regular partition PROTECTED: A backup partition for firmware updates NEEDRESIZE: The user data partition RESERVED: Same as the Reserved field, a reserved partition
Default_Exist	Y (Yes) or N (No), where Y indicates the partition exists.

1.2.4.2 ko Table Format

The ko table is a CSV file that can be edited with a text editor or Excel. For an explanation of each column, please refer to Table 1-6.

Table 1-6. ko table format

Column Header	Description
KO name (*.ko)	Filename, including the .ko extension.
KO path	This field is currently not used.
Vendor/Ramdisk in Normal(vendor/ramdisk)	Can be filled with “vendor” or “ramdisk”, indicating whether the ko is placed in rootfs or ramdisk.
Need loaded before BootTime.completed(Y/N)	This field is currently not used.
Recovery(Y/N)	This field is currently not used.
Build mode	Compilation mode, indicating which specified mode is used. For example, “user/userdebug/eng” means all three modes are used; “userdebug/eng” means only userdebug and eng modes are used, user mode is not used.

1.2.5 Partition Table

Table 1-7 lists the partition table and partition descriptions for MT8676.

Table 1-7. Partition table

Partition_Name	Type	Download_File	Description
preloader_a	Raw data	bl2.img	Bootloader partition
pgpt	Raw data	NONE	GPT partition table
misc	Raw data	NONE	Stores a flag used during OTA upgrade process.
para	Raw data	NONE	Stores system environment information used by LK and kernel.
yocto-expdb	Raw data	NONE	Used to store AEE DB.

Partition_Name	Type	Download_File	Description
nvcfg	EXT4	NONE	Data stored directly through AP side FS by various modules
nvdata	EXT4	NONE	Stores NVRAM data.
protect1	EXT4	NONE	Store SIM lock data.
protect2	EXT4	NONE	Store SIM lock data backup.
seccfg	Raw data	NONE	Stores information related to secure boot.
otp	Raw data	NONE	eMMC OTP (one-time-program, e.g. IMEI)
pstore	Raw data	NONE	Saves some extra logs, such as power off charge.
modem_a	Raw data	modem.img	modem image
spmfw_a	Raw data	spmfw.img	spmfw image
mcf_ota_a	EXT4	mcf_ota.img	mfc_ota image
audio_dsp_a	Raw data	audio_dsp.img	audio_dsp image
pi_img_a	Raw data	pi_img.img	pi_img image
dpm_a	Raw data	dpm.img	dmp image
scp_a	Raw data	scp-fit.img	SCP image
ccu_a	Raw data	ccu.img	CCU image
vcp_a	Raw data	vcp.img	VCP image
sspm_a	Raw data	sspm.img	SSPM image
mcupm_a	Raw data	mcupm.img	MCUPM image
gpueb_a	Raw data	gpueb.img	GPUEB image
apusys_a	Raw data	apusys.img	APUSYS image
boot_a	Raw data	boot.img	Kernel image
tee_a	Raw data	tee.img	ATF image
connsys_bt_a	Raw data	connsys_bt.img	Not used, reserved.
connsys_wifi_a	Raw data	connsys_wifi.img	Not used, reserved.
connsys_gnss_a	Raw data	connsys_gnss.img	GNSS image
logo_a	Raw data	logo.img	logo image
nvrnm	Raw data	NONE	Backup partition for NVDATA
boot_para	Raw data	NONE	Stores DRAM calibration data.
dram_para	Raw data	NONE	Stores DRAM calibration data.
system	EXT4	system.ext4	rootfs image
userdata	EXT4	userdata.ext4	User data
sgpt	Raw data	NONE	GPT partition backup

1.3 Yocto Compilation and Flashing

This section describes the requirements for the host machine to compile Yocto, the instructions for compiling the software and methods for flashing it, as well as the instructions for compiling the SDK and how to use it.

1.3.1 Build Server

Table 1-8 lists the requirements for the compilation host to compile MT8676 Yocto 5.0.

Table 1-8. Build server requirements

Item	Requirement
Disk Space	Not less than 300GB.
Memory	Not less than 32GB.
Distribution Version	Ubuntu 20.04, 22.04, etc., for specific versions refer to Supported Linux Distributions ⁵ .
Build host tools	Refer to Required Packages for the Build Host ⁶ .
Git	Version 1.8.3.1 or higher
tar	Version 1.28 or higher
Python	Version 3.8.0 or higher
GNU Make	Version 4.0 or higher

1.3.2 Compiling Software

The instructions for compiling MT8676 Yocto software are as follows. First, switch to the directory where the codebase is located. The environment variable TEMPLATECONF is used to specify the path where the files bblayers.conf.sample and local.conf.sample are located. The source command is used to initialize the Yocto build environment. This command will create a build directory, copy bblayers.conf.sample and local.conf.sample to the build/conf directory, and rename them to bblayers.conf and local.conf. Note that the script will change the current working directory to build. The final bitbake command starts the compilation. If you need to compile again, just execute the last command. If the terminal is exited and re-logged in, or a new terminal is used, all commands must be executed again.

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676
```

⁵ <https://docs.yoctoproject.org/ref-manual/system-requirements.html#supported-linux-distributions>

⁶ <https://docs.yoctoproject.org/ref-manual/system-requirements.html#required-packages-for-the-build-host>

1.3.3 Software Composition

After the software compilation is complete, the software can be found in the build/tmp/deploy/images/auto8676p1_64 directory. Table 1-9 lists the main image files and their descriptions.

Table 1-9. Image file descriptions

文件名	分区	说明
MT6897_Android_scatter.xml	N/A	Partition table, compiled by ptgen-v2.bb
download_agent/DA_BR.bin	N/A	Download agent, used for flashing. Compiled by collect.bb.
bl2.img	preloader_a preloader_b	Bootloader, compiled by lk2.bb
modem.img	modem_a	Modem firmware, compiled by modem.bb
spmfw.img	spmfw_a	System power management firmware, compiled by collect.bb
mcf_ota.img	mcf_ota_a	Compiled by collect.bb
audio_dsp.img	audio_dsp_a	Audio DSP firmware, compiled by collect.bb
pi_img.img	pi_img_a	Compiled by collect.bb
dpm.img	dpm_a	Compiled by collect.bb
scp-fit.img	scp_a	System Companion Processor firmware, compiled by tinysys-scp.bb
ccu.img	ccu_a	Camera Control Unit firmware, compiled by collect.bb
vcp.img	vcp_a	VCP firmware, compiled by tinysys-vcp.bb
sspm.img	sspm_a	Secure System Power Manager firmware, compiled by collect.bb
mcupm.img	mcupm_a	Power/performance manager firmware, compiled by collect.bb
gpueb.img	gpueb_a	GPU firmware, compiled by tinysys-gpueb.bb
apusys.img	apusys_a	APU firmware, compiled by tinysys-apusys.bb
boot.img	boot_a	Linux kernel in FIT format, including kernel img, dtb, and initramfs. Compiled by linux-mtk-extension_6.1.bb. The initramfs is compiled by core-image-minimal-initramfs.bb.
tee.img	tee_a	ATF, compiled by collect.bb
connsys_bt.img	connsys_bt_a	Reserved partition, not actually used. Compiled by collect.bb.
connsys_wifi.img	connsys_wifi_a	Reserved partition, not actually used. Compiled by collect.bb.
connsys_gnss.img	connsys_gnss_a	GNSS firmware, compiled by collect.bb
logo.img	logo_a	Logo, compiled by makelogo.bb
system.ext4	system	Root filesystem, compiled by mtk-core-image-auto8676.bb
userdata.ext4	userdata	/data partition, compiled by mkusrdata.bbclass

1.3.4 Flashing Software

The flashing software requires a Type-C cable and the SP_Flash_Tool tool. If you do not have the SP_Flash_Tool tool, please visit the [Online](#)⁷ website to download it. After opening the webpage, search for SP_Flash_Tool and download the latest version, as shown in [Figure 1-4](#). After the download is complete, extract the downloaded archive.

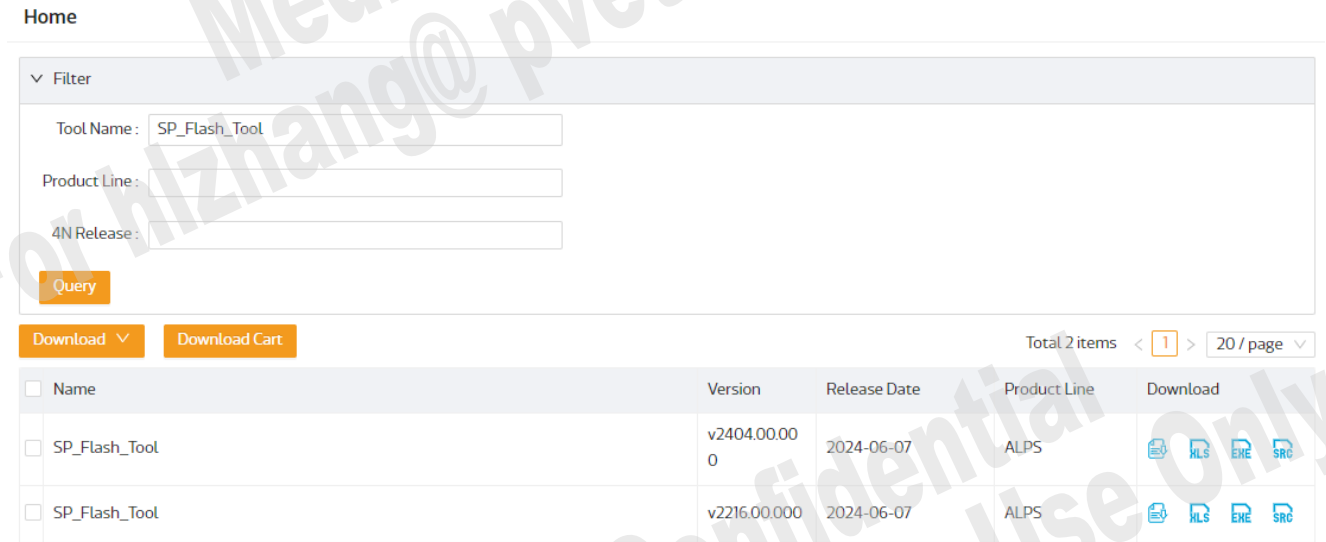


Figure 1-4. Download SP_Flash_Tool

First, connect the platform to the computer using a Type-C cable. Then double-click on SP_Flash_Tool_V6/SPFlashToolV6.exe to open the burning tool. For the first step, select the Download-XML file, click the choose button in the top right corner, and select the download_agent/flash.xml file under the software package. For the second step, select Format All + Download from the dropdown menu below the Download button, as shown in [Figure 1-5](#). For the third step, ensure that the platform is powered off. For the fourth step, click the Download button. For the fifth step, hold down the download key (KPCOLO, SW907) without releasing, power on the platform, and the burning will automatically start, as shown in [Figure 1-6](#). At this point, you can release the download key. If you encounter an error popup, first power off the platform and click the Stop button to exit the download mode. Make sure the platform is completely powered off before attempting to burn again.

⁷ <https://online.mediatek.com/apps/tool/>

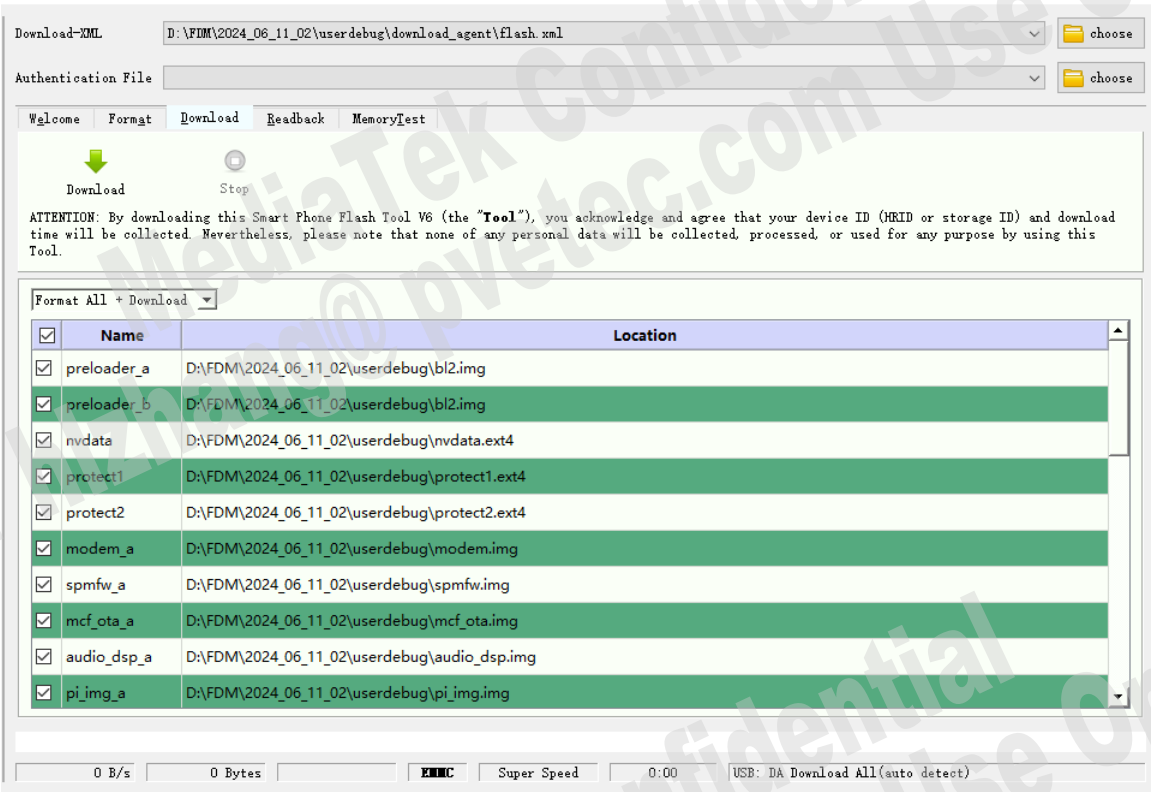


Figure 1-5. Load flash.xml

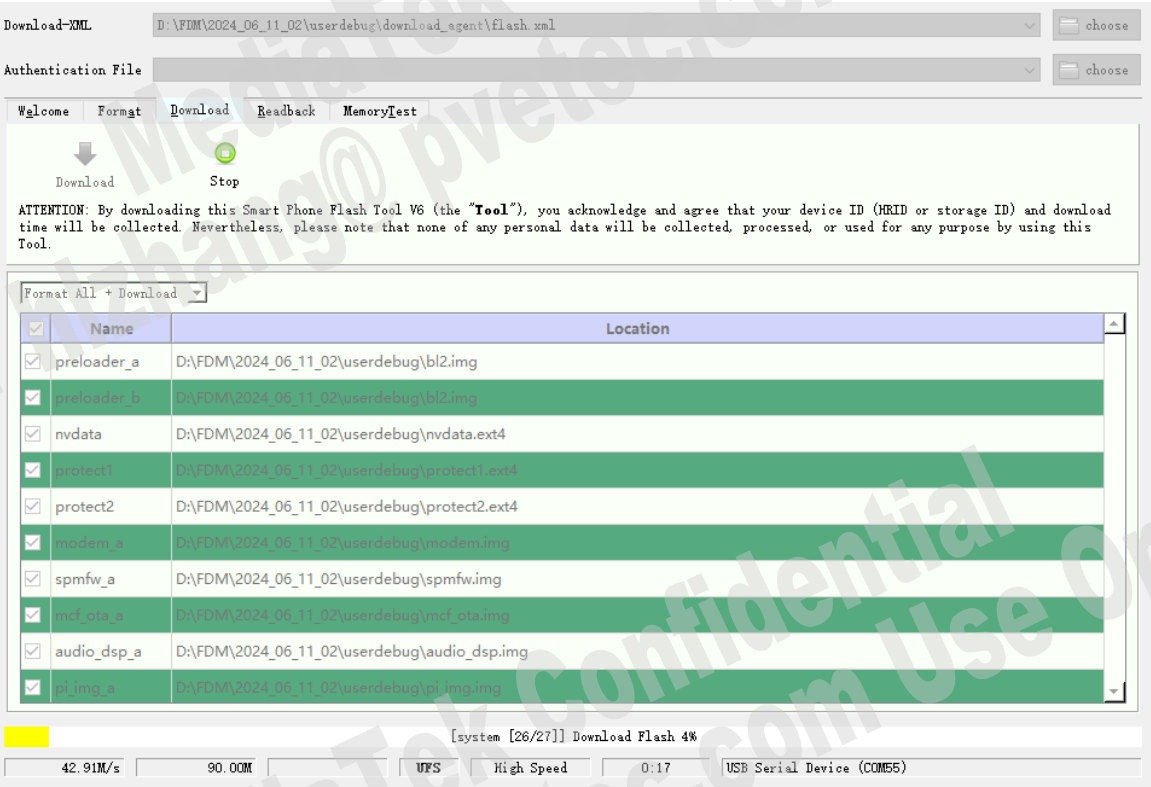


Figure 1-6. Flashing software

1.3.5 Compile SDK

To compile the SDK, use the following command. Compared to compiling software, the bitbake command includes an additional option “-c populate_sdk”.

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676 -c populate_sdk
```

1.3.6 Use SDK

After the SDK is successfully compiled, the installation package can be found in the build/tmp/deploy/sdk directory. For MT8676 Yocto5.0, the name of the installation package is poky-glibc-x86_64-mtk-core-image-auto8676-aarch64-auto8676p1_64-toolchain-5.0.sh.

The installation process is as shown in [Figure 1-7](#). First, clear the environment variable LD_LIBRARY_PATH, as failure to do so will result in the SDK setup failing. The second step is to start the installation script, enter the installation directory for the SDK, input “y” to confirm the installation, and wait for the installation to complete. Follow the prompts from the installation script to execute the setup script, and now you can use the toolchain installed with the SDK.

```
$ unset LD_LIBRARY_PATH
$ ./poky-glibc-x86_64-mtk-core-image-auto8676-aarch64-auto8676p1_64-toolchain-5.0.sh
Poky (Yocto Project Reference Distro) SDK installer version 5.0
=====
Enter target directory for SDK (default: /opt/poky/5.0): /worktmp/codebase/mt8676sdk
You are about to install the SDK to "/worktmp/codebase/mt8676sdk". Proceed [Y/n]? y
Extracting SDK.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
$ . /worktmp/codebase/mt8676sdk/environment-setup-aarch64-poky-linux
$ . /worktmp/codebase/mt8676sdk/environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
$ . /worktmp/codebase/mt8676sdk/environment-setup-aarch64-poky-linux
$ aarch64-poky-linux-gcc --version
aarch64-poky-linux-gcc (GCC) 13.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
mtk19899@mszsdclx1033:~$
```

Figure 1-7. Installing SDK

The SDK installation directory structure, as shown in [Figure 1-8](#), includes the root directory containing environment setup scripts, version information, and other files. The “sysroots/aarch64-poky-linux” directory contains all the executable files, header files, dynamic libraries, static libraries, etc., compiled by MT8676 Yocto, facilitating third-party development in non-Yocto environments. The ELF files in this directory are in aarch64 format. The “sysroots/x86_64-pokysdk-linux” directory contains some local tools, for example, the aarch64 toolchain is located in “sysroots/x86_64-pokysdk-linux/usr/bin/aarch64-poky-linux” directory. There are also many other versions of toolchains available for use, such as aarch64-pokymllib32-linux, arm-poky-linux, etc.

```

├── environment-setup-aarch64-poky-linux
├── environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
├── site-config-aarch64-poky-linux
├── site-config-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
├── sysroots
│   ├── aarch64-poky-linux
│   └── x86_64-pokysdk-linux
│       ├── bin
│       ├── environment-setup.d
│       ├── etc
│       ├── lib
│       ├── sbin
│       ├── usr
│       └── bin
│           ├── .....
│           ├── aarch64-poky-linux
│           ├── aarch64-poky-linux-musl
│           ├── aarch64-pokymllib32-linux
│           ├── aarch64-pokymllib32-linux-musl
│           ├── arm-poky-linux
│           ├── arm-poky-linux-gnueabi
│           ├── arm-poky-linux-musl
│           ├── arm-poky-linux-musleabi
│           ├── arm-pokymllib32-linux
│           ├── arm-pokymllib32-linux-gnueabi
│           ├── arm-pokymllib32-linux-musl
│           ├── arm-pokymllib32-linux-musleabi
│           └── .....
├── version-aarch64-poky-linux
└── version-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi

```

Figure 1-8. SDK installation directory structure

1.4 Yocto Customization

1.4.1 Common Variables

Table 1-10 lists some commonly used variables that are frequently utilized when writing BitBake (.bb) files. There are an extremely large number of variables in Yocto, and merely referring to Table 1-10 is far from sufficient to master Yocto. For a review of all variable values, please refer to Section 1.6.3.

Table 1-10. Yocto common variables

Variable Name	Description
MACHINE	Specifies the target device. Generally defined in local.conf.
MTK_PROJECT	Specifies the project name. Generally defined in <machine>.conf.
PN	Package name. Generally parsed from the filename, for example, for foo_2.6.bb, PN=foo.
PV	Package version. Generally parsed from the filename, for example, for foo_2.6.bb, PV=2.6. If the filename does not specify a version number, such as bar.bb, then PV defaults to 1.0. PV can also be set directly in the bb file.
SRC_URI	Source code URI. Supports protocols such as file, http, https, git, etc.
WORKDIR	bb working directory. All tasks of bb are completed in the working directory.
S	Source code directory. The do_fetch task downloads the source code specified by the SRC_URI variable to the S directory.
B	Build directory. The do_compile task works in this directory.
D	Installation directory. The do_install task installs the products after compilation into this directory.

1.4.2 Basic Structure of a bb File

bb files are Bitbake recipe files, which describe how to build a specific software package, including build dependencies, where to get the source code, how to apply patches, how to compile and install the software, and how to package it. bb files consist of a series of variable definitions and executable tasks. For syntax and operators related to bb files, please refer to the [Bitbake documentation](#)⁸.

The [meta-skeleton](#)⁹ layer provides several example bb files. Taking [hello_1.0.bb](#)¹⁰ as an example, let's introduce the basic structure of a bb file. Initially, it defines some basic information about the software package, such as description, group, license, and the checksum of the license file. This information is communicated to the Bitbake build engine through some standard variables.

```
DESCRIPTION = "Simple helloworld application"
SECTION = "examples"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
```

The SRC_URI variable specifies where to download the source code from, and the S variable specifies the location of the source code. Bitbake supports multiple protocols for downloading source code, such as file, git, http, https, ssh, s3, and more. In this case, the file protocol is used. For the file protocol, Bitbake will look for files in the paths specified by the FILESEXTRAPATHS variable. Similar to the PATH environment variable, the FILESEXTRAPATHS variable consists of several paths separated by colons.

```
SRC_URI = "file://helloworld.c"

S = "${WORKDIR}/sources"
UNPACKDIR = "${S}"
```

The do_compile task is used for compiling the software. Bitbake provides a default empty implementation, so it is necessary to override this task. The names of Bitbake tasks must start with "do_", and they support either shell tasks or Python tasks. The do_compile defined here is a shell task. In shell tasks, variables defined in the bb file can be referenced using the format \${VAR_NAME}.

```
do_compile() {
    ${CC} ${LDFLAGS} helloworld.c -o helloworld
}
```

The do_install task is used to install the software, and the software must be installed into the \${D} directory. Bitbake defines a series of variables for installing different types of files. For example, executable files should be installed into \${bindir}, dynamic libraries into \${libdir}, and configuration files into \${sysconfdir}.

```
do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
}
```

⁸ <https://docs.yoctoproject.org/bitbake/2.8/bitbake-user-manual/bitbake-user-manual-metadata.html>

⁹ <https://git.yoctoproject.org/poky/tree/meta-skeleton>

¹⁰ https://git.yoctoproject.org/poky/tree/meta-skeleton/recipes-skeleton/hello-single/hello_1.0.bb

Sometimes, software packages maintain their own set of configuration, compilation, and installation scripts, eliminating the need to reimplement these in the bb file. For example, packages that use autotools can typically be configured, compiled, and installed using the standard commands “./configure && make && make install”. For these packages, there is no need to write “do_compile” or “do_install” tasks from scratch; instead, you can use the tasks provided by Yocto. Taking [hello_2.10.bb](#)¹¹ as an example, you only need to inherit the “autotools-brokensep” class to compile packages that use autotools. This significantly reduces the burden of writing bb files.

```
DESCRIPTION = "GNU Helloworld application"
SECTION = "examples"
LICENSE = "GPL-3.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=d32239bcb673463ab874e80d47fae504"

SRC_URI = "${GNU_MIRROR}/hello/hello-${PV}.tar.gz"
SRC_URI[sha256sum] = "31e066137a962676e89f69d1b65382de95a7ef7d914b8cb956f41ea72e0f516b"

inherit autotools-brokensep gettext
```

1.4.3 Boot on Startup

Based on the hello_1.0.bb introduced in [Section 1.4.2](#), this section describes how to automatically run a certain program at startup. The MT8676 Yocto uses systemd as the init process, so you first need to write a service file. Below is an example of a helloworld.service file. For more information about systemd services, you can refer to the [systemd documentation](#).

```
[Unit]
Description=hello world service
After=basic.target

[Service]
Type=oneshot
ExecStart=/usr/bin/helloworld

[Install]
WantedBy=multi-user.target
```

Add the service file to the SRC_URI variable.

```
SRC_URI = "file://helloworld.c file://helloworld.service"
```

Since the service file does not require compilation, the do_compile task remains unchanged. In the do_install task, the service file needs to be installed into the directory specified by systemd.

```
do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
    install -d ${D}${systemd_unitdir}/system
    install helloworld.service ${D}${systemd_unitdir}/system
}
```

¹¹ https://git.yoctoproject.org/poky/tree/meta-skeleton/recipes-skeleton/hello-autotools/hello_2.10.bb

Inherit the systemd class, specify the name of the service, and enable boot on startup. The code is as follows. With this, the configuration for boot on startup is complete.

```
inherit systemd
SYSTEMD_PACKAGES = "${PN}"
SYSTEMD_SERVICE:${PN} = "helloworld.service"
```

The complete version of hello_1.0.bb that supports boot on startup is as follows.

```
DESCRIPTION = "Simple helloworld application"
SECTION = "examples"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://helloworld.c file://helloworld.service"

S = "${WORKDIR}/sources"
UNPACKDIR = "${S}"

do_compile() {
    ${CC} ${LDFLAGS} helloworld.c -o helloworld
}

do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
    install -d ${D}${systemd_unitdir}/system
    install helloworld.service ${D}${systemd_unitdir}/system
}

inherit systemd
SYSTEMD_PACKAGES = "${PN}"
SYSTEMD_SERVICE:${PN} = "helloworld.service"
```

1.4.4 Dependencies

When building packages, dependencies can be a headache, but Yocto handles this issue well. If foo.bb depends on the products of bar.bb during compilation, you simply need to add the following code to foo.bb to append the dependent package to the DEPENDS variable. Yocto will ensure that bar is compiled before foo, and during the compilation of foo, it will copy the files installed by bar in \${D} to the working directory of foo (\${WORKDIR}). This allows the compilation of foo to freely use the files installed by bar, such as static libraries, dynamic libraries, header files, etc.

```
# foo.bb
DEPENDS += "bar"
```

If foo.bb depends on bar.bb at runtime, you need to add the following code to foo.bb. Note that the "R" at the beginning of the variable RDEPENDS stands for runtime, and there is an additional "\${PN}", which is necessary.

```
# foo.bb
RDEPENDS:${PN} += "bar"
```

1.4.5 Out-of-tree Drivers

Compiling out-of-tree drivers is a special case because it requires using symbols from the kernel and other out-of-tree drivers, while also providing its own compiled symbols for use by other out-of-tree drivers. Considering that there are many out-of-tree drivers for the MT8676, to avoid numerous dependency issues, a template is provided here. Compiling out-of-tree drivers requires kernel symbols, so it depends on virtual/kernel.

```
DESCRIPTION = "sample out of tree driver"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/GPL-2.0-only;md5=801f80980d171dd6425610833a22dbe6"
DEPENDS = "virtual/kernel"
```

When compiling out-of-tree drivers, it is essential to inherit the module class. This class provides the necessary functions and environment to build kernel modules. In this case, the source code is fetched using the fog protocol, and for more details about the fog protocol, you should refer to [Section 1.4.7](#). Additionally, kernel-toolchain.inc is used to set up the toolchain.

```
# for out-of-tree module bb, must inherit *module*
inherit fog module
SRC_URI = "fog://src/kernel/modules/module-name;name=module-name"
SRCREV_module-name = "${AUTOREV}"

require conf/common/kernel-toolchain.inc

S = "${WORKDIR}/git"
MODULE = "${S}"
LINUX_SRC = "${STAGING_KERNEL_DIR}"
MODULE_NAME = "gpio_sap_ctrl"
```

The variable EXTRA_SYMBOLS is used to specify additional symbol files. Since the MT8676 Linux kernel is divided into common and device module parts, EXTRA_SYMBOLS by default includes the symbol files from the device module. If symbols from other out-of-tree drivers are required, first append the module name to the DEPENDS variable, and then, following the example, append the path of the symbol file to the EXTRA_SYMBOLS variable.

```
# device_module symvers
EXTRA_SYMBOLS += "${TMPDIR}/work-shared/${MACHINE}/device-module-build-artifacts/Module.symvers"

# if you module need other out-of-tree module symbol, please append DEPENDS & EXTRA_SYMBOL
S. below is sample.
# DEPENDS += "oot-module1 oot-module2"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module1/Module.symvers"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module2/Module.symvers"
```

If the Makefile requires additional variables, they can be passed to the Makefile using the export directive. At the same time, there is no need to write the do_compile and do_install tasks yourself. The module class will handle them.

```
# export Makefile required variables. If need more, please add here.
export M="${MODULE}"
export EXTRA_SYMBOLS
export KBUILD_EXTRA_SYMBOLS

# no need write self do_compile / do_install function
# module.bbclass will handle it
```

The complete template is as follows:

```
DESCRIPTION = "sample out of tree driver"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/GPL-2.0-only;md5=801f80980d171dd6425610833a22dbe6"
DEPENDS = "virtual/kernel"

# for out-of-tree module bb, must inherit *module*
inherit fog module
SRC_URI = "fog://src/kernel/modules/module-name;name=module-name"
SRCREV_module-name = "${AUTOREV}"

require conf/common/kernel-toolchain.inc

S = "${WORKDIR}/git"
MODULE = "${S}"
LINUX_SRC = "${STAGING_KERNEL_DIR}"
MODULE_NAME = "gpio_sap_ctrl"

# device_module symvers
EXTRA_SYMBOLS += "${TMPDIR}/work-shared/${MACHINE}/device-module-build-artifacts/Module.symvers"

# if you moudle need other out-of-tree module symbol, please append DEPENDS & EXTRA_SYMBOLS. below is sample.
# DEPENDS += "oot-module1 oot-module2"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module1/Module.symvers"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module2/Module.symvers"

INSANE_SKIP:${PN} += "already-stripped"
INSANE_SKIP:${PN} += "installed-vs-shipped"

# append EXTRA_SYMBOLS to KBUILD_EXTRA_SYMBOLS
python __anonymous () {
    d.appendVar('KBUILD_EXTRA_SYMBOLS', " " + d.getVar('EXTRA_SYMBOLS'))
}

# export Makefile required variables. If need more, please add here.
export M="${MODULE}"
export EXTRA_SYMBOLS
export KBUILD_EXTRA_SYMBOLS

# no need write self do_compile / do_install function
# module.bbclass will handle it
```

1.4.6 image bb

The image bb is used to generate the root file system. Take [core-image-minimal.bb](https://git.yoctoproject.org/poky/tree/meta/recipes-core/images/core-image-minimal.bb)¹² as an example for introduction. The image bb must inherit from the core-image class. The IMAGE_INSTALL variable is used to specify the software packages that need to be installed in the root file system. If you want to install more software packages into the root file system, you just need to append the packages to the IMAGE_INSTALL variable. The image bb for MT8676 is mtk-core-image-auto8676.bb.

```
SUMMARY = "A small image just capable of allowing a device to boot."

IMAGE_INSTALL = "packagegroup-core-boot ${CORE_IMAGE_EXTRA_INSTALL}"

IMAGE_LINGUAS = " "

LICENSE = "MIT"

inherit core-image

IMAGE_ROOTFS_SIZE ?= "8192"
IMAGE_ROOTFS_EXTRA_SPACE:append = "${@bb.utils.contains("DISTRO_FEATURES", "systemd", " + 4096", "", d)}"
```

The file format for the root file system image is specified in <machine>.conf. Taking auto8676p1_64.conf as an example, the format of the root file system image is ext4.

```
# auto8676p1_64.conf
IMAGE_FSTYPES ?= "ext4"
```

1.4.7 Introduction to fog

fog stands for “file or git”, and it is a source code download protocol extended by MediaTek. fog operates on a per-git-repository basis, pulling the entire repository into \${WORKDIR}. The fog:// protocol will convert the source code download protocol to either file:// or git:// based on certain rules.

Here is an example of using fog. First, you need to inherit the fog class, then append the source code path in the fog:// protocol format to the SRC_URI variable. The name parameter must be set and is used to set the name of the module. The SRC_REV variable is used to specify the version of the module, which is generally set to \${AUTOREV}. Finally, the S variable must be set to “\${WORKDIR}/git” or a subdirectory of the git directory, depending on where the build entry point is.

```
inherit fog
SRC_URI = "fog://src/path;name=module-name"
SRCREV_module-name = "${AUTOREV}"

S = "${WORKDIR}/git"
```

fog supports multiple git repositories, as shown in the following example. Compared to a single git repository, you must specify the destsuffix parameter, which is used to designate the subdirectory where the repository is located. You also

¹² <https://git.yoctoproject.org/poky/tree/meta/recipes-core/images/core-image-minimal.bb>

need to set multiple entries in SRC_URI. Additionally, SRCREV_FORMAT is introduced, which is used to set the format of SRCREV.

```
inherit fog
SRC_URI="fog://src/path;name=module-name1;destsuffix=git/sub-dir1"
SRC_URI="fog://src/path;name=module-name2;destsuffix=git/sub-dir2"
SRC_URI="fog://src/path;name=module-name3;destsuffix=git/sub-dir3"
SRCREV_module-name1 = "${AUTOREV}"
SRCREV_module-name2 = "${AUTOREV}"
SRCREV_module-name3 = "${AUTOREV}"
SRCREV_FORMAT="module-name1_module-name2_module-name3"

S = "${WORKDIR}/git"
```

1.4.8 Create Project

Table 1-11 lists the files that need to be created for setting up a new project. It is recommended to copy from auto8676p1_64 and then modify the relevant files. Alternatively, you can use the one-click creation script meta/meta-mediatek-mt8676/scripts/create_project.sh and follow the prompts to operate.

Table 1-11. Create project

Item	Example
machine.conf	meta/meta-mediatek-mt8676/conf/machine/auto8676p1_64.conf
templates	meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64/bblayers.conf.sample meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64/local.conf.sample
dtb	src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/auto8676p1_64.dts
defconfig	src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/configs/auto8676p1_64_defconfig
partition table	meta/meta-mediatek-mt8676/recipes-bsp/ptgen/files/ auto8676p1_64/partition_table.csv
ko table	meta/meta-mediatek-mt8676/recipes-kernel/linux/ko_order_table/auto8676p1_64/ko_order_table.csv
fstab	meta/meta-mediatek-mt8676/recipes-core/base-files/base-files/auto8676p1_64/fstab
collect-bins	prebuilt/bsp/collect-bins/mt8676/auto8676p1_64
dws	src/devtools/dct/dws/mt6897/auto8676p1_64.dws
lk2	src/bsp/lk2/project/auto8676p1_64.mk
scp	src/tinysys/mt8676/scp/project/RV55_A/mt6897/auto8676p1_64

1.4.9 Create Meta Layer

A Yocto meta layer is essentially a directory containing a conf/layer.conf file, and it is recommended that the directory name starts with “meta-”, such as meta-foo, meta-bar, etc. Therefore, to create a meta layer, you first need to create a conf/layer.conf file. Taking the creation of meta-foo as an example, the sample layer.conf is as follows. BBPATH adds the current layer path to the Bitbake search path, BBFILES specifies the path where the bb files are located, BBFILE_COLLECTIONS assigns a unique identifier to the current layer, BBFILE_PATTERN sets the directory prefix for the bb files, and BBFILE_PRIORITY sets the priority of the bb files, with higher numbers indicating higher priority. If two layers

have bb files with the same name, the one with the higher priority is used. LAYERSERIES_COMPAT is used to set the Yocto versions supported by the layer.

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":{LAYERDIR}"

# We have a recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
           ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "foo"
BBFILE_PATTERN_foo = "^${LAYERDIR}/"
BBFILE_PRIORITY_foo = "10"

LAYERSERIES_COMPAT_foo = "scarthgap"
```

Figure 1-9 shows the directory structure of a meta layer. In addition to the conf/layer.conf file, the classes directory is used to store bbclass files, and the conf/machine and conf/templates directories are used to store project-related files. Directories starting with “recipes-” are used to store bb and bbappend files. The structure of these directories depends on the BBFILES variable, which is generally set to “\${LAYERDIR}/recipes-*/*/*.bb \${LAYERDIR}/recipes-*/*/*.bbappend”.

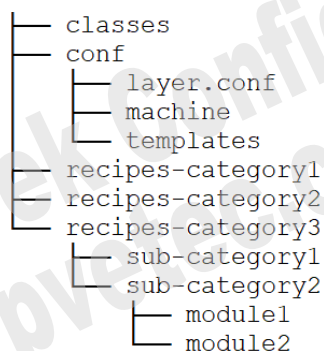


Figure 1-9. Meta layer directory structures

1.5 Yocto Build Mode

Yocto does not define a build mode by default, and can only compile one type of software. However, during the project cycle, different software are needed at different stages. MediaTek has proposed a solution similar to Android for DEV/SQC/MP. Table 1-12 lists the differences between the various build modes.

Table 1-12. Yocto build mode

Build Mode	Android Config Setting	Yocto Config Setting	Usage
user	1. Disable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....) 2. Disable debug features (AEE/UART/ADB/...)	1. Disable debug configs (TFA/LK2.0/Kernel/TEE/APP/....) 2. Disable debug features (AEE/UART/ADB/...)	1. Products MP 2. SQC performance test
userdebug	1. Disable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....)	1. Disable debug configs (TFA/LK2.0/Kernel/TEE/APP/....)	1. SQC main test 2. RD DEV/UT/IT test

Build Mode	Android Config Setting	Yocto Config Setting	Usage
	2. Enable debug features (AEE/UART/ADB/...)	2. Enable debug features (AEE/UART/ADB/...)	
eng	1. Enable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....) 2. Enable debug features (AEE/UART/ADB/...)	1. Enable debug configs (TFA/LK2.0/Kernel/TEE/APP/....) 2. Enable debug features (AEE/UART/ADB/...)	1. RD debug

1.5.1 Yocto Build Mode Usage

MediaTek Yocto uses the environment variable `TARGET_BUILD_VARIANT` to distinguish between three build modes. The value of the environment variable can be set to `user`, `userdebug`, or `eng`, which are used to compile user software, `userdebug` software, and `eng` software, respectively. If the environment variable is not set, or if the value is not one of `user`, `userdebug`, or `eng`, then `userdebug` software will be compiled.

To differentiate these three modes in the `bb` files, MediaTek appends different values to the variables `DISTRO_FEATURES` and `OVERRIDES` based on the value of the environment variable `TARGET_BUILD_VARIANT`, as shown in [Table 1-13](#).

Table 1-13. Build variant

TARGET_BUILD_VARIANT	DISTRO_FEATURES	OVERRIDES
user	variant-user	variant-user
userdebug	variant-userdebug	variant-userdebug
eng	variant-eng	variant-eng

The following are the instructions for compiling `eng` software. If you need to compile `user` or `userdebug` software, you only need to change the value of the environment variable `TARGET_BUILD_VARIANT`.

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
export TARGET_BUILD_VARIANT=eng
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676
```

1.5.2 Yocto Build Mode Adaptation

Modules need to be adapted to the build mode in order to compile the actual `user`, `userdebug`, or `eng` software. First, it is necessary to define the differences between these three modes, and then make targeted modifications to the `bb` files for adaptation. Here are two examples to illustrate how to adapt.

Taking the Linux kernel (`linux-mtk-extension_6.1.bb`) as an example, the Linux kernel uses `defconfig` for configuration. `User` software only needs to use the project `defconfig` file, `userdebug` software uses both the project `defconfig` and the `userdebug.config` files, and `eng` software uses the project `defconfig` and the `eng.config` files. The relevant adaptation code is as follows. The variable `VARIANT_CONFIG` defines the additional config file, which is empty by default. If it's `userdebug`,

then it is set to `userdebug.config`, and similarly for `eng`. Here, Yocto's conditional variable syntax is used. Finally, a piece of code is prepended before the `do_configure` task to merge the two config files using the `merge_config.sh` script.

```
VARIANT_CONFIG = ""
VARIANT_CONFIG:variant-eng = "${S}/${KCONFIG_EXT_PREFIX}/kernel/configs/eng.config"
VARIANT_CONFIG:variant-userdebug =
"${S}/${KCONFIG_EXT_PREFIX}/kernel/configs/userdebug.config"

do_configure:prepend() {
    bbnote "VARIANT_CONFIG is ${VARIANT_CONFIG}"

    install -d ${B}/arch/${KERNEL_ARCH}/configs/
    KCONFIG_CONFIG=${B}/arch/${KERNEL_ARCH}/configs/${KBUILD_CONFIG} \
        ${S}/scripts/kconfig/merge_config.sh -m \
        -r ${MODULEDIR}/arch/${KERNEL_ARCH}/configs/${KBUILD_CONFIG} \
        ${VARIANT_CONFIG}
    ...
}
```

Using `adb` (`android_tools_5.1.1.r37.bbappend`) as an example, user software requires `adb` to be disabled, while `userdebug` and `eng` software require `adb` to be enabled. The relevant adaptation code is as follows. Here, Yocto's conditional variable syntax is used to clear the `SYSTEMD_SERVICES` and `FILES` variables only when building user software, and to append instructions to the `do_install` task to delete the related files.

```
# variant user disable adbd
SYSTEMD_SERVICE:${PN}-adbd:variant-user = ""
FILES:${PN}-adbd:variant-user = ""
do_install:append:variant-user() {
    rm -rfv "${D}${bindir}/adbd"
    rm -rfv "${D}${systemd_unitdir}/system/android-tools-adbd.service"
}

# avoid build fail
INSANE_SKIP:${PN} += "installed-vs-shipped"
```

1.6 Yocto Debugging Techniques

1.6.1 Build Directory Structure

After the software compilation is complete, the directory structure is shown in [Figure 1-10](#). Compared to the original directory structure, there are additional directories: `build`, `downloads`, and `sstate-cache`. The `build` directory is specified by the variable `TOPDIR` and contains intermediate files from the compilation, images, RPM packages, and other files. For information about the `downloads` directory and `sstate-cache`, please refer to [Section 1.6.4](#).

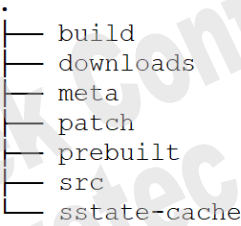


Figure 1-10. Directory structure after compilation

The build directory contains a lot of information, and being familiar with the build directory is helpful for debugging Yocto. The variable TOPDIR points to the build directory. Table 1-14 lists the commonly used files and directories in the build directory.

Table 1-14. Commonly used files and directories in the build directory

File or Directory	Description
build/conf/local.conf	Copied from local.conf.sample.
build/conf/bblayers.conf	Copied from bblayers.conf.sample.
build/conf/templateconf.cfg	Contains the value of the environment variable TEMPLATECONF.
build/tmp/deploy/images/<project>	Contains image files for flashing.
build/tmp/deploy/rpm	Contains RPM packages generated after bb compilation.
build/tmp/deploy/license	Subdirectories are formatted as <PN>, containing license information for the corresponding bb.
build/tmp/sysroots-components	Subdirectories are formatted as <ARCH>/<PN>, containing files installed to the rootfs by the corresponding bb.
build/tmp/work	Subdirectories are formatted as <ARCH>/<PN>/<PV>, corresponding to the work directory of bb. All tasks of bb are completed in the work directory. For details, please refer to Section 1.6.2.
build/tmp/work-shared	Shared work directory. If files need to be shared between bbs, they can be placed in this directory.

1.6.2 Commonly Used Subdirectories in the Work Directory

The work directory is located under build/tmp/work, and all tasks of a bb (BitBake recipe) are completed in this work directory. The variable WORKDIR points to the work directory of the bb. Table 1-15 lists the commonly used subdirectories in the work directory. By utilizing the files in the work directory, you can quickly clarify issues. For example, if do_compile fails, you can check the temp/run.do_compile and log.do_compile files; to confirm which package a file has been packaged into, you can check the packages-split directory.

Table 1-15. Commonly used subdirectories in the work directory

File or Directory	Description
git	Stores the source code specified by the SRC_URI variable. Some bbs place the source code in a \${PN}-\${PV} subdirectory. The S variable points to this directory.
build	Directory for storing intermediate and final compilation files. The B variable points to this directory. Sometimes the B variable is equal to the S variable.
recipe-sysroot	Stores files of the packages specified by the DEPENDS variable. Header files and libraries are searched for in this directory during compilation.
recipe-sysroot-native	Similar to the recipe-sysroot directory, it stores files of the “*-native” packages specified by the DEPENDS variable, including local tools used during compilation, such as cross-compilers.
image	Stores files installed by the do_install task. The D variable points to this directory.
package	Stores the final packaged files.
packages-split	Similar to the package directory, but divided into subdirectories by package name. Generally, in addition to the default \${PN} package, Yocto also adds several packages like \${PN}-dbg, \${PN}-dev, \${PN}-doc, etc.
sysroot-destdir	The files in this directory are a subset of those in the image directory. If they are depended upon by another bb, the files in this directory will be copied to the recipe-sysroot or recipe-sysroot-native directory of the other bbs.
temp	Contains task scripts and running logs. For example, for the do_compile task, run.do_compile is the task script, and log.do_compile is the task log. The log.task_order file records the execution time and order of tasks.

1.6.3 Printing Variable Values

Yocto has a vast number of variables, and the variable operators in Yocto are very flexible. Additionally, there are many configuration files, which makes it difficult to determine the final value of a variable. We can use the “-e” option of bitbake to print out the process of all variable changes as well as their final values, which can greatly improve the efficiency of debugging. Take the WORKDIR variable of systemd as an example. Execute the following command to save the environment variables of systemd to a file named systemd.env. Since there are so many variables, redirecting and saving to a file makes analysis easier.

```
bitbake systemd -e > systemd.env
```

When you open the systemd.env file, you can search and find content similar to the following. You can see that the WORKDIR variable is manipulated twice: the first time, its value is set on line 407 of bitbake.conf; the second time, its value for the tag doc is set on line 467 of documentation.conf. Finally, the value of the WORKDIR variable is printed. Using this method, you can easily locate which configuration file has performed what operation on the variable.

```
#
# $WORKDIR [2 operations]
#   set /mt8676/codebase/meta/poky/meta/conf/bitbake.conf:407
#     "${BASE_WORKDIR}/${MULTIMACH_TARGET_SYS}/${PN}/${PV}"
#   set /mt8676/codebase/meta/poky/meta/conf/documentation.conf:467
#     [doc] "The pathname of the working directory in which the OpenEmbedded build system
# builds a recipe. This directory is located within the TMPDIR directory structure and changes
# as different packages are built."
# pre-expansion value:
#   "${BASE_WORKDIR}/${MULTIMACH_TARGET_SYS}/${PN}/${PV}"
WORKDIR="/mt8676/codebase/build/tmp/work/aarch64-poky-linux/systemd/255.4"
```

1.6.4 Accelerating Yocto Compilation

Starting from scratch, Yocto compilation is very slow, mainly in two aspects. First, Yocto will download a large number of source packages and code repositories. Second, many unmodified software packages will also be compiled from scratch. You can save the intermediate files generated after the Yocto compilation is completed to speed up the next compilation. After the Yocto compilation is completed, the directories downloads and sstate-cache will be generated. The downloads directory contains the downloaded source packages and code repositories, specified by the variable DL_DIR; the sstate-cache directory contains the compilation cache, specified by the variable SSTATE_DIR.

Before the first compilation, create the file build/conf/site.conf and add the following code. This line of code will package the downloaded code repositories into a compressed file. After the compilation is completed, copy and save the downloads and sstate-cache directories. Suppose they are saved to the "/path/to/your/saved/downloads/" directory and the "/path/to/your/saved/sstate-cache/" directory, respectively.

```
BB_GENERATE_MIRROR_TARBALLS = "1"
```

Before the second compilation, create the file build/conf/site.conf and add the following code. SOURCE_MIRROR_URL points to the saved "downloads" directory, and SSTATE_MIRRORS points to the saved "sstate-cache" directory (note that there is a fixed "PATH" at the end). If the .bb file has not been modified, Yocto will directly take the results of the previous compilation from the SSTATE_MIRRORS directory. If the sstate cache does not hit, and the software package needs to be downloaded for compilation, Yocto will prioritize obtaining the software package from SOURCE_MIRROR_URL.

```
SOURCE_MIRROR_URL ?= "file:///path/to/your/saved/downloads/"
INHERIT += "own-mirrors"
BB_GENERATE_MIRROR_TARBALLS = "1"

SSTATE_MIRRORS = "file://.* file:///path/to/your/saved/sstate-cache/PATH"
BB_SIGNATURE_HANDLER = "OEBasicHash"
BB_HASHSERVE = ""
```

It should be noted that after the .bb file is modified, the sstate cache will become invalid, so the sstate cache should be updated regularly. The "downloads" directory generally does not become invalid. If you do not want to use the sstate cache, you can add the "--no-setsccache" option to bitbake.

1.7 Boot Process

The boot process of the MT8676 Yocto is shown in Figure 1-11. It mainly consists of four major parts: Bootloader, kernel, initramfs, and systemd.

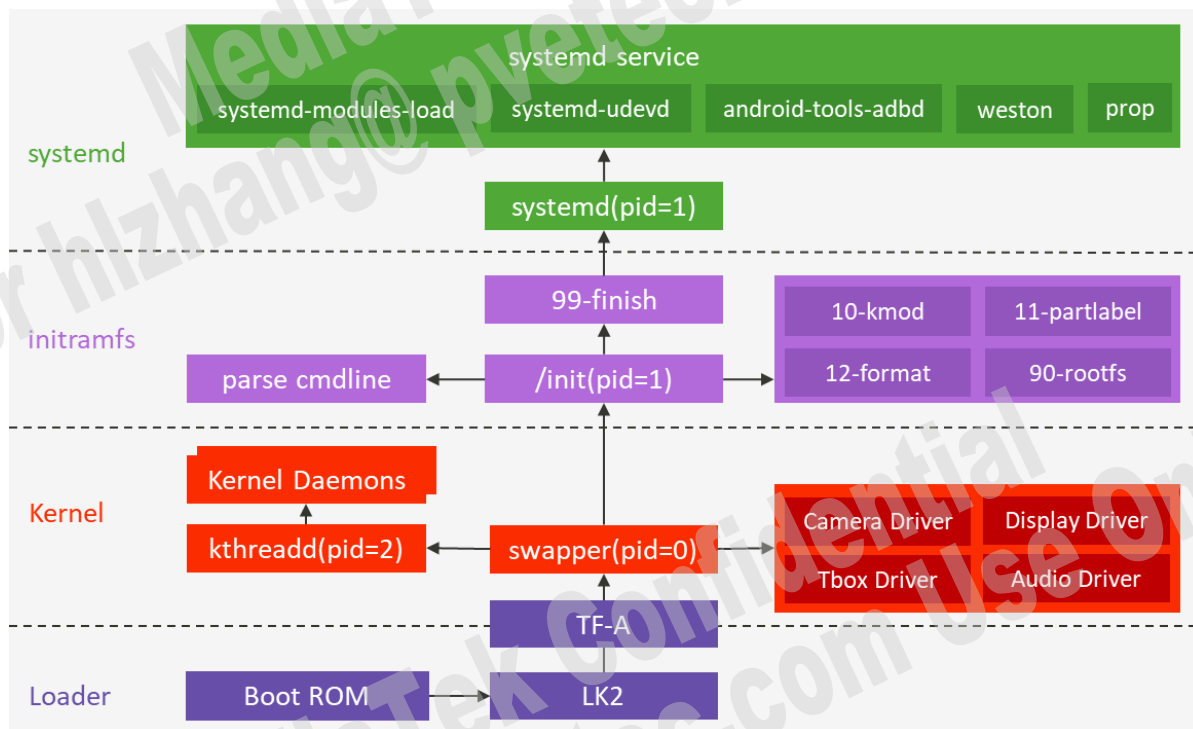


Figure 1-11. Boot process

1.7.1 BootROM

BootROM is a hardware device (similar to SRAM/DRAM), containing a segment of factory-installed, unmodifiable, and solidified program. After power-on, the BootROM starts executing the pre-set bootcode internally, loads the bootloader LK2 into SRAM, and then executes it. The main functions are as follows:

- Boot and load LK2 into SRAM for execution.
- When no valid image or code is detected, BootROM guides the system into download mode.
- Perform some verification work.
- Hardware initialization, such as serial ports, flash, etc.

1.7.2 LK2

LK2, also known as the bootloader, runs in SRAM (which does not require initialization). The main functions of LK2 are as follows:

- Initialize DRAM and various peripherals.

- Load TF-A (Trusted Firmware-A) and jump to TF-A for initialization.
- Load the kernel.
- Load tinysys (such as GPUUEB, SCP, VCP, etc.).
- Jump to the kernel.

1.7.3 Kernel

The kernel is the core component of an operating system, responsible for managing computer hardware and software resources and providing basic services for the operating system and applications. The kernel is the first component loaded by the operating system and runs in memory as the core of the entire system, controlling all system resources such as CPU, memory, input/output, file systems, etc. The kernel provides low-level services and interfaces for applications to call and use.

Kernel Assembly Phase (starting from ENTRY(_stext)):

1. Set to SVC mode and disable all interrupts.
2. Obtain CPU ID.
3. Verify the device tree blob (dtb).
4. Create page table entries.
5. Configure the r13 register, which sets the function to jump to after enabling the MMU.
6. Enable the MMU.
7. Jump to start_kernel, entering the C phase.

Kernel C Phase (starting from kernel_init): This phase mainly completes the initialization work related to the hardware platform. After the related initialization is finished, it executes the init program.

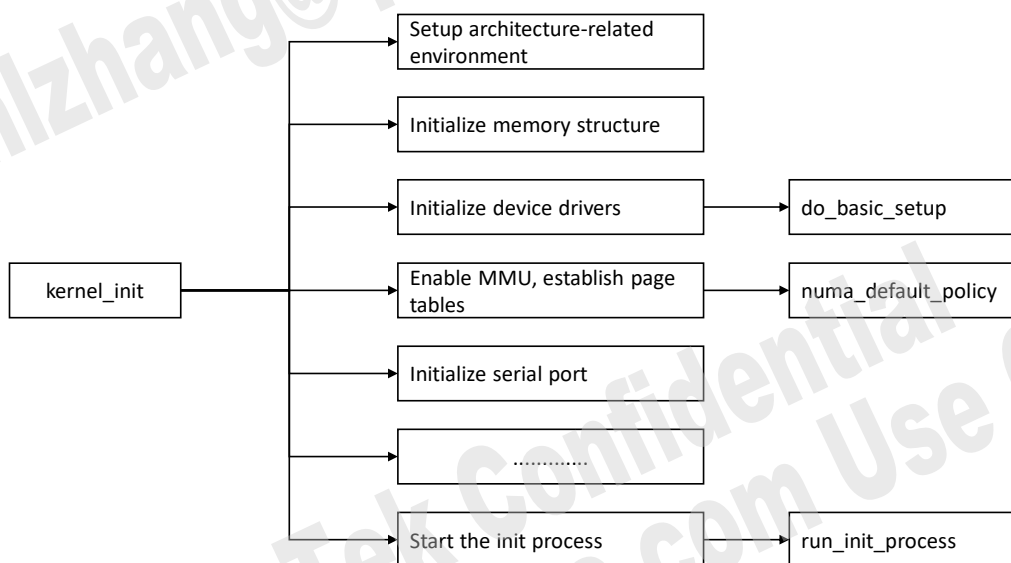


Figure 1-12. kernel_init

1.7.4 initramfs

initramfs is an intermediate layer between the kernel and systemd. Its primary function is to load some basic kernel modules to prepare for switching the root filesystem (rootfs) and executing systemd. After the kernel initialization is completed, the init script under initramfs is run first. The init script will parse the kernel boot parameters and export them as shell environment variables. Then it calls the initialization scripts in the /init.d directory. On the MT8676 platform, the /init.d directory contains scripts such as 10-kmod, 11-partlabel, 12-format, 90-rootfs, 99-finish, etc.

10-kmod loads the kernel modules specified in the /etc/modules.list file. The /etc/modules.list is generated based on the ko table file. If the “vendor/Ramdisk in Normal(vendor/ramdisk)” column is ramdisk, then the /etc/modules.list file will include this kernel module. For the format of the ko table, please refer to [Section 1.2.4.1](#). The /etc/modules.list should contain some basic kernel modules, such as the UFS driver, etc.

11-partlabel is responsible for parsing the names of partitions and creating symbolic links in /dev/disk/by-partlabel. Scripts like 12-format and 90-rootfs depend on these symbolic links.

12-format is responsible for formatting partitions such as nvdata, protect1, protect2, etc.

90-rootfs is responsible for mounting the rootfs partition to the /rootfs directory. The rootfs partition is specified by the kernel boot parameter “root=PARTLABEL=system”.

99-finish is responsible for mounting procfs, sysfs, devfs in the /rootfs directory and switching the root directory to /rootfs. Finally, it starts systemd.

1.7.5 systemd

systemd is a system and service manager for Linux systems. Compared to the traditional SysVinit, systemd offers modern features such as parallel startup, dependency management, and on-demand activation.

After systemd takes control, it first performs a series of pre-initialization tasks, such as setting default environment variables and initializing the logging system. It then reads and parses unit files located in directories like /etc/systemd/system and /lib/systemd/system. These files define the configuration for services, mount points, devices, and other resources.

Based on the dependencies described in the unit files, systemd constructs a dependency tree. Then, systemd attempts to reach the default target specified by /etc/systemd/system/default.target, which is composed of a set of interdependent units. Systemd will start the services that the default target depends on in parallel and ensure that the services start in the correct order according to the dependency tree.

Table 1-16. Common services

Service	Description
systemd-modules-load	Reads configuration files under /etc/modules-load.d and loads kernel modules. The file /etc/modules-load.d/10-kernel-modules.conf includes vendor kernel modules from the ko

Service	Description
	table.
systemd-udev	Responsible for managing the creation and removal of device nodes.
android-tools-adbd	ADB (Android Debug Bridge) daemon
weston	Weston desktop service
prop	Property service process. The tools getprop and setprop interact with this service.
mdpd_8676	MDP background service process
camerahalservice	Camera service process
thermal-daemon	Thermal daemon process

1.8 Common Debugging Tools in Yocto

Table 1-17. Common debugging tools in Yocto

Tool	Description
log	The primary source of information for analyzing all issues. For information about the Yocto log system, please refer to the document <i>MTK_Yocto_Log_Tool_Guide</i> , which can be downloaded in Online .
AEE	When an exception occurs, AEE collects relevant debugging information into a db (datafile) file. It is a powerful tool for debugging platform abnormal reboots, kernel exceptions, user program exceptions, etc. Please refer to the document <i>MTK_Yocto_AEE_Tool_Guide</i> , which can be downloaded in Online .
kasan	Used for debugging kernel memory leaks, please use the auto8676p1_64_kasan project. In addition to kasan, there are other tools for debugging memory leaks such as kmemleak, asan, valgrind, etc.
ftrace	Captures kernel trace logs to analyze performance issues.
systemd-analyze	Captures the startup time of systemd services to analyze boot time.
strace	Captures system calls made by a program.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.